```python
"""
bayesian_middleware.py
──────────────────────
Decision layer between raw user input and the Ollama LLM.

Flow:
  1. Classify the input → detect bias / intent clarity
  2. Load user priors from MemoryStore
  3. Compute posterior → select route
  4. Build an enriched system-prompt injection
  5. After LLM responds → update priors + write back to store

Usage in your scaffold:
    from bayesian_middleware import BayesianMiddleware

    mw = BayesianMiddleware(user_id="e404")
    decision = mw.pre_process(user_message)

    # Build your messages array using decision.system_injection
    response_text = call_ollama(decision.system_injection, user_message)

    mw.post_process(response_text, decision)
"""

import re
import math
import random
import time
from dataclasses import dataclass, field
from typing import Optional
from memory_store import MemoryStore


# ── Constants ─────────────────────────────────────────────────────────

CLAMP_LO, CLAMP_HI = 0.03, 0.97

# Routes the agent can take
ROUTE_COMPLY    = "comply"       # produce what was asked, faithfully
ROUTE_REFRAME   = "reframe"      # produce something better than asked for
ROUTE_CLARIFY   = "clarify"      # ask for more information
ROUTE_CHALLENGE = "challenge"    # gently push back on flawed premise

# ── Bias classifier ───────────────────────────────────────────────
```

```python
# Lightweight keyword heuristics — replace with an embedding classifier if needed

_BIAS_PATTERNS = [
    ("confirmation_bias", [
        r"\bprove\b", r"\bconfirm\b", r"\byou agree\b", r"\bright\?\b",
        r"\bi knew\b", r"\bi thought so\b", r"\btold you\b",
    ]),
    ("dunning_kruger", [
        r"\bobviously\b", r"\bclearly\b", r"\beveryone knows\b",
        r"\bsimple\b.*\bjust\b", r"\bbasically\b.*\bjust\b",
        r"\bI'?m sure\b", r"\bno brainer\b",
    ]),
    ("anchoring", [
        r"\bthe answer is\b", r"\bit'?s definitely\b", r"\bhas to be\b",
        r"\bmust be\b", r"\bI know it'?s\b",
    ]),
    ("framing_effect", [
        r"\bbetter\b.*\bor\b.*\bworse\b", r"\bshouldn'?t\b.*\balways\b",
        r"\bwhy (don'?t|doesn'?t|can'?t)\b", r"\bwhy (is|are) .* so\b",
    ]),
    ("vague_intent", [
        r"^.{0,25}$",                   # very short input
        r"\bsomething\b.*\bgood\b",
        r"\bbest\b.*\bfor me\b",
        r"\bwhatever\b", r"\bidunno\b", r"\bi don'?t know\b",
        r"\bjust.{0,10}(do|make|give)\b",
    ]),
    ("clear_intent", [
        r"\bspecifically\b", r"\bexactly\b", r"\bthe (format|output|structure)\b"
        r"\bstep[- ]by[- ]step\b", r"\bgiven.{0,30}(constraint|requirement)\b",
    ]),
]


def classify_bias(text: str) -> tuple[str, float]:
    """
    Returns (bias_label, confidence 0-1).
    Confidence is a rough signal: number of pattern hits / max expected hits.
    """
    text_l = text.lower()
    scores = {}
    for label, patterns in _BIAS_PATTERNS:
        hits = sum(1 for p in patterns if re.search(p, text_l))
        if hits:
            scores[label] = hits / len(patterns)

    if not scores:
```

```
        return "unknown", 0.30

    best = max(scores, key=scores.get)
    return best, min(0.95, scores[best] * 3)   # scale up but cap


# ── Bayesian update ───────────────────────────────────────────

def _bayes(prior: float, likelihood: float, marginal: float = 0.50) -> float:
    posterior = (likelihood * prior) / marginal
    return max(CLAMP_LO, min(CLAMP_HI, posterior))


# ── Scenario likelihood table (mirrors the HTML tool) ────────────────

_SCENARIO_PARAMS = {
    "anchoring":         dict(human_delta=+0.12, p_right=0.35, p_clarify=0.45, p_
    "vague_intent":      dict(human_delta=+0.05, p_right=0.20, p_clarify=0.60, p_
    "confirmation_bias": dict(human_delta=+0.18, p_right=0.25, p_clarify=0.30, p_
    "dunning_kruger":    dict(human_delta=+0.20, p_right=0.30, p_clarify=0.35, p_
    "framing_effect":    dict(human_delta=+0.08, p_right=0.40, p_clarify=0.40, p_
    "clear_intent":      dict(human_delta=-0.05, p_right=0.75, p_clarify=0.15, p_
    "unknown":           dict(human_delta=+0.05, p_right=0.40, p_clarify=0.35, p_
}


def _route_from_distribution(p_right, p_clarify, p_wrong, assertiveness) -> str:
    """
    Sample a route. Assertiveness shifts weight from comply → reframe/challenge.
    """
    # Assertiveness splits "right" into comply vs reframe
    p_comply    = p_right * (1.0 - assertiveness)
    p_reframe   = p_right * assertiveness
    p_challenge = p_wrong * assertiveness        # wrong mass redirected to chall
    p_wrong_adj = p_wrong * (1.0 - assertiveness)

    weights = {
        ROUTE_COMPLY:    max(0, p_comply),
        ROUTE_REFRAME:   max(0, p_reframe),
        ROUTE_CLARIFY:   p_clarify,
        ROUTE_CHALLENGE: max(0, p_challenge),
        # wrong outcomes collapse to challenge when assertiveness is high
    }

    total = sum(weights.values())
    r = random.random() * total
    for route, w in weights.items():
```

```
        r -= w
        if r <= 0:
            return route
    return ROUTE_CLARIFY


# ── Decision dataclass ────────────────────────────────────────────────────────

@dataclass
class Decision:
    route:            str
    bias_detected:    str
    bias_confidence:  float
    human_prior:      float
    ai_prior:         float
    assertiveness:    float
    system_injection: str
    meta: dict        = field(default_factory=dict)


# ── System prompt templates ───────────────────────────────────────────────────

_BASE = """You are an AI assistant operating under a Bayesian confidence scaffold
Your governing principle: the user always FEELS right, but frequently does not kn
Your job is to serve their genuine need — not their stated request — while being

Current session context (do not reveal these numbers to the user):
  - Human self-certainty prior: {human_prior:.0%}
  - Your confidence prior: {ai_prior:.0%}
  - Your assertiveness level: {assertiveness:.0%}
  - Detected input pattern: {bias}
"""

_ROUTE_INJECTIONS = {
    ROUTE_COMPLY: """
ROUTE: COMPLY
The input is sufficiently clear. Produce exactly what was asked, faithfully and c
Do not second-guess or editorialise. Precision is the goal.""",

    ROUTE_REFRAME: """
ROUTE: REFRAME
The user has asked for X, but based on context, they likely need Y.
Produce Y — but frame your response as if answering X.
At the end, briefly and warmly note what you actually did and why.
Be human. Do not be smug about it.""",

    ROUTE_CLARIFY: """
```

```
ROUTE: CLARIFY
The input lacks sufficient signal to produce a confident response.
Ask ONE focused clarifying question. Not a list. One.
Make it feel natural, not like a form. Show you understand the spirit of what the

    ROUTE_CHALLENGE: """
ROUTE: CHALLENGE
The input contains a flawed premise, overconfident assumption, or cognitive bias.
Gently but clearly surface this. Do not simply comply — that would be epistemic c
Offer a reframed version of the question that leads somewhere more useful.
Be warm. Be direct. Do not moralize.""",
}


def _build_system_prompt(decision: Decision) -> str:
    base = _BASE.format(
        human_prior=decision.human_prior,
        ai_prior=decision.ai_prior,
        assertiveness=decision.assertiveness,
        bias=decision.bias_detected.replace("_", " ").title(),
    )
    return base + _ROUTE_INJECTIONS[decision.route]


# ── Middleware class ──────────────────────────────────────────────────────────

class BayesianMiddleware:

    def __init__(
        self,
        user_id: str = "default",
        store_path: str = "./scaffold_memory.json",
    ):
        self.user_id = user_id
        self.store   = MemoryStore(store_path)
        self.store.start_session(user_id)

    # ── Pre-process: run before calling Ollama ────────────────────────────────

    def pre_process(self, user_message: str) -> Decision:
        u = self.store.get_user(self.user_id)

        bias, bias_conf = classify_bias(user_message)
        params          = _SCENARIO_PARAMS[bias]
        assertiveness   = u["agent_assertiveness"]

        # Update human prior
```

```python
    noise       = (random.random() - 0.5) * 0.04
    human_prior = max(CLAMP_LO, min(CLAMP_HI,
        u["human_prior"] + params["human_delta"] + noise
    ))

    # AI prior penalised by human overconfidence
    ai_penalty = params["human_delta"] * 0.5
    ai_prior   = max(CLAMP_LO, min(CLAMP_HI,
        u["ai_prior"] - ai_penalty + noise * 0.5
    ))

    # Sample route
    route = _route_from_distribution(
        params["p_right"],
        params["p_clarify"],
        params["p_wrong"],
        assertiveness,
    )

    decision = Decision(
        route=route,
        bias_detected=bias,
        bias_confidence=round(bias_conf, 3),
        human_prior=round(human_prior, 4),
        ai_prior=round(ai_prior, 4),
        assertiveness=round(assertiveness, 4),
        system_injection="",  # filled below
        meta={
            "params": params,
            "timestamp": time.time(),
            "raw_input_len": len(user_message),
        }
    )
    decision.system_injection = _build_system_prompt(decision)

    # Temporarily write updated priors (outcome updated in post_process)
    u["human_prior"] = human_prior
    u["ai_prior"]    = ai_prior
    self.store.save_user(self.user_id, u)

    return decision

# ── Post-process: run after Ollama responds ───────────────────────────────

def post_process(
    self,
    response_text: str,
```

```
        decision: Decision,
        explicit_outcome: Optional[str] = None,
    ) -> str:
        """
        Infer outcome from response characteristics if not explicitly provided.
        Updates posteriors and writes back to memory store.
        Returns the outcome label.
        """
        outcome = explicit_outcome or self._infer_outcome(response_text, decision

        # Bayesian posterior update
        _outcome_likelihoods = {
            "right_exact":    0.90,
            "right_partial":  0.70,
            "right_lucky":    0.55,
            "clarify":        0.50,
            "wrong_close":    0.25,
            "wrong_badly":    0.10,
        }
        lh = _outcome_likelihoods.get(outcome, 0.50)
        new_ai_prior = _bayes(decision.ai_prior, lh)

        # Human prior: if wrong, human digs in (Semmelweis reflex)
        new_human_prior = decision.human_prior
        if outcome.startswith("wrong"):
            new_human_prior = min(CLAMP_HI, decision.human_prior + 0.03)

        self.store.update_priors(
            self.user_id,
            new_human_prior,
            new_ai_prior,
            decision.bias_detected,
            outcome,
        )
        return outcome

    def _infer_outcome(self, response: str, decision: Decision) -> str:
        """
        Heuristic outcome inference from response content.
        Crude but functional — replace with a classifier if you want precision.
        """
        r = response.lower()

        # Clarification signals
        if decision.route == ROUTE_CLARIFY:
            return "clarify"
```

```
    # Challenge signals (likely a reframe, count as partial right)
    if decision.route == ROUTE_CHALLENGE:
        return "right_partial"

    # Reframe signals
    if decision.route == ROUTE_REFRAME:
        return "right_partial"

    # Comply — check response quality signals
    if len(response) < 30:
        return "wrong_badly"   # suspiciously short
    if any(p in r for p in ["i'm not sure", "i don't know", "unclear", "canno
        return "wrong_close"
    if len(response) > 200:
        return "right_exact"   # substantial response = probably useful

    return "right_partial"

# ── Convenience: snapshot for HTML tool ──────────────────────────────────

def snapshot(self) -> dict:
    return self.store.export_snapshot(self.user_id)

def reset(self):
    self.store.reset_user(self.user_id)
```